

不过需要注意的是，上界分析的结果在趋势上能反映算法的效率，但有两个不精确性：一是公式本身的不精确性。例如，“非主流”基本操作的影响、隐藏在大 O 记号后的低次项和最高项系数；二是对程序实现细节与计算机硬件的依赖性，例如，对复杂表达式的优化计算、把内存访问方式设计得更加“cache 友好”等。在不少情况下，算法实际能解决的问题规模与表 8-1 所示有着较大差异。

尽管如此，表 8-1 还是有一定借鉴意义的。考虑到目前主流机器的执行速度，多数算法竞赛题目所选取的数据规模基本符合此表。例如，一个指明 $n \leq 8$ 的题目，可能 $n!$ 的算法已经足够， $n \leq 20$ 的题目需要用到 2^n 的算法，而 $n \leq 300$ 的题目可能必须用至少 n^3 的多项式时间算法了。

8.2 再谈排序与检索

假设有 n 个整数，希望把它们按照从小到大的顺序排列，应该怎样做呢？也许你会说：调用 STL 中的 `sort` 或者 `stable_sort` 即可。可是读者们有没有想过：这些现成的排序函数是怎样工作的呢？

8.2.1 归并排序

第一种高效排序算法是归并排序。按照分治三步法，对归并排序算法介绍如下。

划分问题：把序列分成元素个数尽量相等的两半。

递归求解：把两半元素分别排序。

合并问题：把两个有序表合并成一个。

前两部分是很容易完成的，关键在于如何把两个有序表合成一个。图 8-2 演示了一个合并的过程。每次只需要把两个序列的最小元素加以比较，删除其中的较小元素并加入合并后的新表即可。由于需要一个新表来存放结果，所以附加空间为 n 。

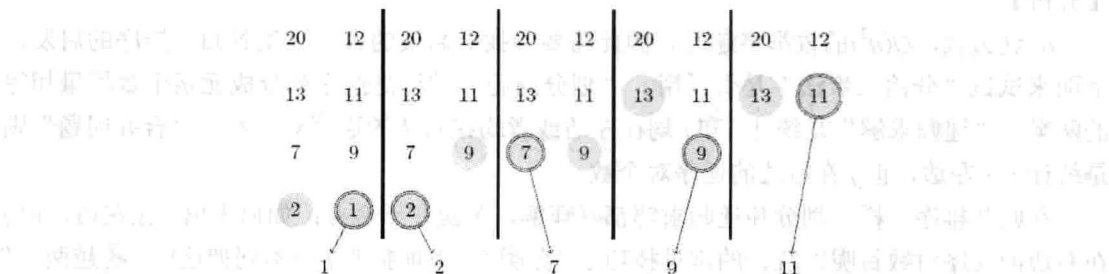


图 8-2 合并过程：时间是线性的，需要线性的辅助空间

这个过程极为重要，希望读者仔细体会。代码如下：

程序 8-4 归并排序（从小到大）

```
void merge_sort(int* A, int x, int y, int* T){
    if(y-x > 1){
```

```

int m = x + (y-x)/2;           //划分
int p = x, q = m, i = x;
merge_sort(A, x, m, T);        //递归求解
merge_sort(A, m, y, T);        //递归求解
while(p < m || q < y){
    if(q >= y || (p < m && A[p] <= A[q])) T[i++] = A[p++];
    //从左半数组复制到临时空间
    else T[i++] = A[q++];        //从右半数组复制到临时空间
}
for(i = x; i < y; i++) A[i] = T[i];    //从辅助空间复制回 A 数组
}
}

```

代码中的两个条件是关键。首先，只要有一个序列非空，就要继续合并（`while(p<m||q<y)`），因此在比较时不能直接比较 $A[p]$ 和 $A[q]$ ，因为可能其中一个序列为空，从而 $A[p]$ 或者 $A[q]$ 代表的是一个实际不存在的元素。正确的方式是：

- 如果第二个序列为空（此时第一个序列一定非空），复制 $A[p]$ 。
- 否则（第二个序列非空），当且仅当第一个序列也非空，且 $A[p] \leq A[q]$ 时，才复制 $A[p]$ 。

上面的代码巧妙地利用短路运算符“`||`”把两个条件连接在了一起：如果条件 1 满足，就不会计算条件 2；如果条件 1 不满足，就一定会计算条件 2。这样的技巧很实用，请读者细心体会。另外，读者如果仍然不太习惯 $T[i++] = A[p++]$ 这种“复制后移动下标”的方式，是时候把它们看懂、弄熟了。

不难看出，归并排序的时间复杂度和最大连续和的分治算法一样，都是 $O(n \log n)$ 的。

逆序对问题。给一系列数 $a_1, a_2, \dots, a_n$ ，求它的逆序对数，即有多少个有序对 (i, j) ，使得 $i < j$ 但 $a_i > a_j$ 。 n 可以高达 10^6 。

【分析】

n 这么大， $O(n^2)$ 的枚举将超时，因此需要寻找更高效的方法。受到归并排序的启发，下面来试试“分治三步法”是否适用。“划分问题”过程是把序列分成元素个数尽量相等的两半；“递归求解”是统计 i 和 j 均在左边或者均在右边的逆序对个数；“合并问题”则是统计 i 在左边，但 j 在右边的逆序对个数。

和归并排序一样，划分和递归求解都好理解，关键在于合并：如何求出 i 在左边，而 j 在右边的逆序对数目呢？统计的常见技巧是“分类”。下面按照 j 的不同把这些“跨越两边”的逆序对进行分类：只要对于右边的每个 j ，统计左边比它大的元素个数 $f(j)$ ，则所有 $f(j)$ 之和便是答案。

幸运的是，归并排序可以“顺便”完成 $f(j)$ 的计算：由于合并操作是从小到大进行的，当右边的 $A[j]$ 复制到 T 中时，左边还没来得及复制到 T 的那些数就是左边所有比 $A[j]$ 大的数。此时在累加器中加上左边元素个数 $m-p$ 即可（左边所剩的元素在区间 $[p, m]$ 中，因此元素个数为 $m-p$ ）。换句话说，在代码上的唯一修改就是把“`else T[i++] = A[q++];`”改成“`else`

{ T[i++] = A[q++]; cnt += m-p; }”。当然，在调用之前应给 cnt 清零。

提示 8-7：归并排序的时间复杂度为 $O(n\log n)$ 。对该算法稍加修改，可以统计序列中的逆序对的个数，时间复杂度不变。

8.2.2 快速排序

快速排序是最快的通用内部排序算法。它由 Hoare 于 1962 年提出，相对归并排序来说不仅速度更快，并且不需辅助空间（还记得那个 T 数组吗）。按照分治三步法，将快速排序算法作如下介绍。

划分问题：把数组的各个元素重排后分成左右两部分，使得左边的任意元素都小于或等于右边的任意元素。

递归求解：把左右两部分分别排序。

合并问题：不用合并，因为此时数组已经完全有序。

读者也许会觉得这样的描述太过笼统，但事实上，快速排序本来就不是只有一种实现方法。“划分过程”有多个不同的版本，导致快速排序也有不同版本。读者很容易在互联网上找到各种快速排序的版本，这里不再给出代码。

快速选择问题。输入 n 个整数和一个正整数 k ($1 \leq k \leq n$)，输出这些整数从小到大排序后的第 k 个（例如， $k=1$ 就是最小值）。 $n \leq 10^7$ 。

【分析】

选择第 k 大的数，最容易想到的方法是先排序，然后直接输出下标为 $k-1$ 的元素（别忘了 C 语言中数组下标从 0 开始），但 10^7 的规模即使对于 $O(n\log n)$ 的算法来说较大。有没有更快的方法呢？

答案是肯定的。假设在快速排序的“划分”结束后，数组 $A[p \cdots r]$ 被分成了 $A[p \cdots q]$ 和 $A[q+1 \cdots r]$ ，则可以根据左边的元素个数 $q-p+1$ 和 k 的大小关系只在左边或者右边递归求解。可以证明，在期望意义下，程序的时间复杂度为 $O(n)$ 。

提示 8-8：快速排序的时间复杂度为：最坏情况 $O(n^2)$ ，平均情况 $O(n\log n)$ ，但实践中几乎不可能达到最坏情况，效率非常高。根据快速排序思想，可以在平均 $O(n)$ 时间内选出数组中第 k 大的元素。

8.2.3 二分查找

排序的重要意义之一，就是为检索带来方便。试想有 10^6 个整数，希望确认其中是否包含 12345，最容易想到的方法就是把它们放到数组 A 中，然后依次检查这些整数是否等于 12345。这样的方式对于“单次询问”来说运行得很好，但如果需要找 10000 个数，就需要把整个数组 A 遍历 10000 次。而如果先将数组 A 排序，就可以查找得更快——好比在字典中查找单词不必一页一页翻一样。

在有序表中查找元素常常使用二分查找（Binary Search），有时也译为“折半查找”，基本思路就像是“猜数字游戏”：你在心里想一个不超过 1000 的正整数，我可以保证在 10

次之内猜到它——只要你每次告诉我猜的数比你想要的大一些、小一些，或者正好猜中。

猜的方法就是“二分”。首先我猜 500，除了运气特别好正好猜中之外^①，不管你说“太大”还是“太小”，我都能把可行范围缩小一半：如果“太大”，那么答案在 1~499 之间；如果“太小”，那么答案在 501~1000 之间。只要每次选择可行区间的中点去猜，每次都可以把范围缩小一半。由于 $\log_2 1000 < 10$ ，10 次一定能猜到。

这也是二分查找的基本思路。

提示 8-9：逐步缩小范围法是一种常见的思维方法。二分查找便是基于这种思路，它遵循分治三步法，把原序列划分成元素个数尽量接近的两个子序列，然后递归查找。二分查找只适用于有序序列，时间复杂度为 $O(\log n)$ 。

尽管可以用递归实现，但一般把二分查找写成非递归的：

程序 8-5 二分查找（迭代实现）

```
int bsearch(int* A, int x, int y, int v){
    int m;
    while(x < y) {
        m = x+(y-x)/2;
        if(A[m] == v) return m;
        else if(A[m] > v) y = m;
        else x = m+1;
    }
    return -1;
}
```

上述 while 循环常常直接写在程序中。二分查找常常用在一些抽象的场合，没有数组 A，也没有要查找的 v，但是二分的思想仍然适用。

提示 8-10：二分查找一般写成非递归形式。

下面提一个有趣的问题：如果数组中有多个元素都是 v，上面的函数返回的是哪一个的下标呢？第一个？最后一个？都不是。不难看出，如果所有元素都是要查找的，它返回的是中间那一个。有时，这样的结果并不是很理想，能不能求出值等于 v 的完整区间呢（由于已经排好序，相等的值会排在一起）？

下面的程序，当 v 存在时返回它出现的第一个位置。如果不存在，返回这样一个下标 i：在此处插入 v（原来的元素 A[i], A[i+1], … 全部往后移动一个位置）后序列仍然有序。

程序 8-6 二分查找求下界

```
int lower_bound(int* A, int x, int y, int v){
    int m;
    while(x < y){
```

^① 如果没有公证人，你可以不动声色地换一个数。